

Programación I - Primer Semestre 2026

Trabajo Práctico: Super Elizabeth Sis

Las Autoridades de la UNGS nos encargaron desarrollar un video juego similar al Super Mario Bros pero con algunas importantes diferencias que mencionamos a continuación.

El malvado Rey Camir secuestró a Mario, el plomero de la princesa, en su castillo. La princesa Elizabeth transformada ahora en nuestra heroína se lanza a combatir a las hordas de secuaces del rey para poder rescatarlo (ver Figura 1).

El objetivo de este trabajo práctico es crear un juego en el cual la princesa Elizabeth vaya avanzando en un escenario, en el que aparecen distintos enemigos que tiene que eliminar, con el objetivo de llegar con vida al castillo.



Figura 1: Ejemplo del juego.

El Juego: Super Elizabeth Sis

Requerimientos obligatorios

1. **Inicio del juego:** Al iniciar el juego, la princesa Elizabeth debe aparecer en el medio de la pantalla. El escenario debe consistir en un mapa horizontal que vaya avanzando a medida que la princesa se mueva hacia la derecha. En el final del mapa debe aparecer un castillo. Ver Figuras 1 y 2.

2. **Islas flotantes:** El mapa deberá contar con una serie de islas donde la princesa pueda moverse libremente. Las islas del nivel más bajo (el piso) deben tener un tamaño fijo y una separación entre ellas que permita que la princesa pueda caer al vacío. Las otras islas flotantes deberán ubicarse a distintas alturas (por lo menos 2 niveles de altura diferentes) y deben contar con algún nivel de aleatoriedad, para que ninguna ejecución del juego tenga el mismo mapa que la anterior.
3. **Movimiento:** La princesa deberá poder moverse hacia la derecha y hacia la izquierda presionando las flechas correspondientes. Cuando la princesa se mueva hacia la derecha, la princesa debe avanzar sobre el mapa y el mapa debe moverse hacia la izquierda, de manera que se vaya avanzando hasta llegar al final del mismo. En ningún caso la princesa puede sobrepasar los límites de la pantalla.
4. **Salto y gravedad:** Si se presiona un botón de salto (por ejemplo la flecha hacia arriba) la princesa deberá poder saltar hasta cierta altura máxima o hasta que su cabeza choque con una isla flotante. La princesa no debe atravesar las islas con su salto. Cuando la princesa no esté sobre una isla, deberá estar en caída libre. Este estado de caída constante que simula la gravedad, debe detenerse cuando los pies de la princesa hagan contacto con alguna isla.
5. **Enemigos:** Deberán aparecer enemigos voladores desde ambos lados (derecho e izquierdo) de la pantalla, de manera aleatoria. Los enemigos deberán avanzar hacia el lado opuesto al que aparecieron. Los enemigos pueden superponerse entre sí y pero no pueden atravesar las islas. Una vez que los enemigos avancen lo suficiente para salirse de la pantalla, deben volverse null. En todo momento, el juego debe tener una cantidad mínima de enemigos vivos en pantalla a elección del grupo.
6. **Ataque de los enemigos:** Cuando los enemigos voladores hagan contacto con la princesa, deberán quitarle 1 vida y morir al mismo tiempo. Todo enemigo que muera debe desaparecer (volverse null).
7. **Disparo:** La princesa puede realizar un disparo para matar a los enemigos. Cuando se presione el botón izquierdo del mouse, la princesa podrá lanzar un proyectil desde su posición hacia la posición del cursor del mouse al momento del click (Ver Figura 2). Dicho proyectil deberá matar al primer enemigo con el cual colisione (volviéndose ambos objetos null). Si el proyectil no hace contacto con ningún enemigo, deberá volverse null al momento de salir de los límites de la pantalla. Solo debe existir 1 disparo como máximo en todo momento. Es decir, que no se puede volver a disparar mientras exista un proyectil en vuelo.



Figura 2: Ejemplo disparo desde princesa.

8. **Condición de victoria:** Al final del mapa, en el límite derecho debe existir un castillo. Al hacer contacto con el castillo, el juego debe terminar y darse por ganado.
9. **Condición de derrota:** La princesa debe tener una cantidad fija de vidas que se deberá mostrar en pantalla en todo momento. Cada vez que haga contacto con un enemigo y cada vez que se caiga por el vacío, pierde una vida. Al llegar a 0 vidas, se debe terminar el juego y darse por perdido.
10. **Buen diseño:** El código del proyecto deberá tener un buen diseño de modo que cada objeto tenga bien delimitadas sus responsabilidades. Aclaraciones: cuando un objeto ya no esta más debe ser eliminado (null), no vale únicamente ocultar las imágenes del juego. Para la implementación de los enemigos se debe usar arreglos de objetos, no se permite ninguna otra estructura de datos.

Requerimientos opcionales

La implementación a entregar debe cumplir como mínimo con todos los requerimientos obligatorios planteados arriba, pero si el grupo lo desea, puede implementar nuevos elementos para enriquecer la aplicación. Sugerimos a continuación algunas ideas:

- Distintos tipos de ataque: la princesa puede tener otro tipo de ataque. Por ejemplo: un disparo secundario que explote en un radio al hacer contacto con un enemigo, pudiendo dañar a varios; un ataque tipo rayo en dirección del mouse y que mate todo enemigo que lo toque, etc.
- Items de regalo: Al matar enemigos puede haber una chance de dejar un item de regalo. Este item puede ser una vida extra, un ataque extra que se active durante un tiempo, o un escudo, etc.
- Distintos tipos de enemigos: Pueden diseñarse distintos tipos de enemigos, que hagan más daño a la princesa, tengan ataques a la distancia, etc.

- Jefe final: al entrar al castillo, en vez de ganar el juego, se debe enfrentar a un jefe final, el cual tiene su propia cantidad de vidas y sus propios ataques a distancia.

Informe solicitado

Además del código, la entrega debe incluir un documento en el que se describa el trabajo realizado, que debe incluir, **obligatoriamente y como mínimo**, las siguientes secciones:

Carátula: Una carátula del documento con nombres, apellidos, legajos, y direcciones de email de cada integrante del equipo.

Introducción: Una breve introducción describiendo el trabajo práctico.

Descripción: Una explicación general de cada clase implementada, describiendo las variables de instancia y dando una breve descripción de cada método. También deben incluirse los problemas encontrados, las decisiones que tomaron para poder resolverlos y las soluciones encontradas.

Implementación: Una sección de implementación donde se incluya el código fuente correctamente formateado y comentado, si corresponde.

Conclusiones: Algunas reflexiones acerca del desarrollo del trabajo realizado y de los resultados obtenidos. Pueden incluirse lecciones aprendidas durante el desarrollo del trabajo.

Condiciones de entrega

El trabajo práctico se debe hacer en grupos de **exactamente tres** personas.

El nombre del proyecto de Eclipse debe tener el siguiente formato: los apellidos en minúsculas de los tres integrantes, separados con guiones, seguidos de **-tp-p1**.
Por ejemplo, **amaya-benelli-castro-tp-p1**.

Cada grupo deberá entregar tanto el informe como el código fuente del trabajo práctico.

La modalidad de entrega es mediante repositorio en Git: asegurarse de que el repositorio sea privado y que el nombre del proyecto de Git tenga los apellidos en minúsculas de los tres integrantes, separados con guiones, seguidos del string **-tp-p1**.

Por ejemplo, **amaya-benelli-castro-tp-p1**. Consultar el username de cada docente y agregar a los docentes como **maintainer's**. El informe del trabajo práctico en formato pdf puede incluirse directamente en el repositorio.

Fecha de entrega: Consultar en el moodle de cada comisión la fecha y horario límite.